Ders 13 Genişletilmiş Notu: Operatör Aşırı Yükleme, Interface'ler, Enum'lar, Generic Sınıflar ve ArrayList

Ders 13 Genişletilmiş Notu: Operatör Aşırı Yükleme, Interface'ler, Enum'lar, Generic Sınıflar ve ArrayList

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı ve Java'da Olmayan Kavramlar
- 2. Operatör Aşırı Yükleme: Java'da Karmaşık Sayı Toplama
- 3. C#'ta Operatör Tanımlama
- 4. C++, Python ve Operatör Aşırı Yüklemenin Sınırları
- 5. Çoklu Kalıtım
- 6. Nesnelerde == ve equals
- 7. Interface Kelimesinin Gündelik Anlamı
- 8. Java'da Interface: Tanım, UML ve implements
- 9. Interface Neden Kullanılır, Kuralları Nelerdir?
- 10. Slayt Örneği: Araç Vergisi
- 11. Kod Örneği: IDatabase
- 12. Enum'lar
- 13. Generic Sınıflar ve Generic Metotlar
- 14. Veri Yapısı Nedir?
- 15. Bağlı Liste
- 16. ArrayList
- 17. Koleksiyonların Sınıf Hiyerarşisi
- 18. Kod Örneği: Course ve Student

1. Dersin Planı ve Java'da Olmayan Kavramlar

Şimdiye kadar sınıf ve nesne kavramlarını, sınıflar ve nesneler arasındaki ilişkileri ve soyut sınıfları gördük. Hoca bu dersin planını şöyle kurdu: Önce nesneye yönelik programlamaya ait olan ama Java'da bulunmayan iki kavrama, operatör aşırı yüklemeye (operator overloading) ve çoklu kalıtıma (multiple inheritance) bakılacak. Sonra dersin asıl konuları gelecek: interface'ler, enum'lar, generic sınıflar ve bunların üzerinden Java'daki hazır veri yapılarının kullanımı. Java'da olmayan bir kavramı daha önce de görmüştük: copy constructor.

Hoca bu kavramları neden anlattığını şöyle açıkladı: İşi tersinden bilmek, yani bir dilde neyin olmadığını bilmek de bazen daha doğrudur. Aynı sorunun başka dillerde nasıl çözüldüğünü görmek için Java'nın yanında C#, C++ ve Python örneklerine de bakıldı.

Soru: Neden hep dizi kullanıyoruz, `ArrayList` gibi yapıları neden kullanmıyoruz? **Cevap:** Generic'lere gelince göreceğiz. `ArrayList` ve `LinkedList` bir sonraki bölümün konusu. Bilenlerin kullanmasında bir sakınca yok.

2. Operatör Aşırı Yükleme: Java'da Karmaşık Sayı Toplama

2.1 Toplama operatörü neyi toplar?

Nesneye yönelik programlamada çoğunlukla kendi tasarladığımız sınıflar ve nesneler üzerinde işlem yaparız. Bir operatörün bu nesneler için yeniden tanımlanmasına operatör aşırı yükleme denir.

Hoca önce şunu sordu: `+` işareti neyi toplar? Sınıfla birlikte şu liste çıktı: iki `int`, iki `float`, bir `float` ile bir `int` toplanabilir, karakterler toplanabilir, `String`'ler birleştirilebilir (concatenation). Yani `+` genel olarak ilkel tipler üzerinde çalışır, `String` de bunun özel bir durumudur. Ama bir kitap nesnesiyle başka bir kitap nesnesini toplayamayız.

> **Ders sonrası not:** Derste bir öğrenci `float` ile `int` toplanınca sonucun `double`'a döndüğünü söyledi; doğrusu sonuç `float`'tır. Karakter toplamının türü de `char` değil, `int`'tir: `'a' + 'b'` ifadesinin çalışma zamanındaki türü `Integer` çıktı ve iki `char` değişkeninin toplamını `char`'a atamak derleme hatası verdi (javac ile denendi).

2.2 Complex sınıfı ve derleme hatası

Hoca bu sınırı görmek için NetBeans'te iki `private` alanı olan bir `Complex` sınıfı yazdı: gerçel kısım `real` ve sanal kısım `imaginary`. Sınıfa, nesne oluşturulurken bu iki değeri alan bir constructor eklendi, sonra `c1 = new Complex(1, 2)` ve `c2 = new Complex(2, 2)` nesneleri üretildi.

Nesneleri ekrana yazdırmak için `toString` metodu hatırlatıldı. Bu metot `Object` sınıfında tanımlıdır ve biz onu override ederek istediğimiz gibi değiştirebiliriz.

Soru: `toString` ne yapıyordu? **Cevap:** Bir öğrenci nesnenin özelliklerini yazdırdığını söyledi. Hoca iki noktayı düzeltti: `toString` yazdırmaz, bir `String` döndürür. Özellikleri içermek zorunda da değildir, içine ne yazarsak onu döndürür.

NetBeans'in otomatik ürettiği `toString` kodunda hoca ilginç bir ayrıntı gösterdi: Son karakteri tek bir karakter olarak alıp `+` ile önceki `String`'lere ekliyordu. Çıktı `a + bi` biçimine getirildi, böylece `1 + 2i` ve `2 + 2i` görüldü.

Sonra asıl soru geldi: Bu iki sayıyı şöyle toplayabilir miyim?

```
Complex c3 = c1 + c2;   // derlenmez
```

Derleyici bunu kabul etmez. `c3` önceden `new Complex(4, 4)` ile oluşturulup `c3 = c1 + c2;` yazılsa da aynı hata alınır. Hata mesajı, iki işlenenin de türünün `Complex` olduğunu söyler:

```
error: bad operand types for binary operator '+'
  first type:  Complex
  second type: Complex
```

Bu mesaj derste IDE'de görüldü, aynı mesaj javac ile de alındı (javac ile denendi).

2.3 Birinci çözüm: statik metot

Soru: Realleri mi toplayacağız, imajinerleri mi? **Cevap:** Amaç karmaşık sayı toplamaktır, yani ikisi de ayrı ayrı toplanır.

Bir öğrenci çözümü önerdi: İki karmaşık sayı alıp toplayan `add` adlı bir metot yazmak. İki karmaşık sayının toplamı yine bir karmaşık sayıdır, o yüzden metot `Complex` döndürür. Statik metotlar nesne üretmeden çağrılabildiği için ilk sürüm statik yazıldı. Metodun içinde yeni bir `Complex` nesnesi üretmek gerektiği için bir de parametresiz constructor eklendi. Bu constructor `this(0, 0)` ile diğer constructor'ı çağırır, böylece aynı kod iki kez yazılmaz.

Burada önemli bir ayrıntı var: `real` ve `imaginary` alanları `private` olduğu halde metodun içinde `c1.real` yazabiliyoruz, çünkü bu kod aynı sınıfın içindedir. Başka bir sınıfta `c1.real` görünmez.

Derste yazılan kodun altyazıdan kurulmuş hali aşağıdadır. Hocanın sonradan eklediği ikinci çözüm ve `equals` da aynı sınıftadır.

```java
public class Complex {
    private int real, imaginary;

    public Complex() {
        this(0, 0);
    }

    public Complex(int real, int imaginary) {
        this.real = real;
        this.imaginary = imaginary;
    }

    // 1. çözüm: statik metot
    public static Complex add(Complex c1, Complex c2) {
        Complex c = new Complex();
        c.real = c1.real + c2.real;
        c.imaginary = c1.imaginary + c2.imaginary;
        return c;
    }

    // 2. çözüm: nesne metodu, yeni nesne döndürür
    public Complex add(Complex c1) {
        Complex c = new Complex();
        c.real = this.real + c1.real;
        c.imaginary = this.imaginary + c1.imaginary;
        return c;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final Complex other = (Complex) obj;
        if (this.real != other.real) {
            return false;
        }
        if (this.imaginary != other.imaginary) {
            return false;
        }
        return true;
    }

    @Override
    public String toString() {
        return "Complex{" + real + " + " + imaginary + "i}";
    }

    public static void main(String[] args) {
        Complex c1 = new Complex(1, 2);
        Complex c2 = new Complex(2, 2);
        System.out.println(c1);
        System.out.println(c2);
```

```
        Complex c3 = Complex.add(c1, c2);
        Complex c4 = c1.add(c2);
        Complex c5 = c2.add(c1);
        System.out.println(c3);
        System.out.println(c4);
        System.out.println(c5);

        System.out.println(c4 == c5);
        System.out.println(c4.equals(c5));
        c2 = c1;
        System.out.println(c1 == c2);
    }
}
```

Çıktı (javac ile derlenip çalıştırıldı):

```
Complex{1 + 2i}
Complex{2 + 2i}
Complex{3 + 4i}
Complex{3 + 4i}
Complex{3 + 4i}
false
true
true
```

Hocanın sayısal örneği şöyle çözülür. Verilen: $c_1 = 1 + 2i$ ve $c_2 = 2 + 2i$. Gerçel kısımlar toplanır: $1 + 2 = 3$. Sanal kısımlar toplanır: $2 + 2 = 4$. Sonuç: $c_3 = 3 + 4i$.

Hoca bu çözümü şöyle özetledi: Operatörle yapamadığım işi bir fonksiyonla yaptım. `c3` burada yalnızca bir referanstır, nesne metottan gelir.

2.4 İkinci çözüm: nesne metodu

İkinci tasarımda metot statik değildir ve bir nesne üzerinden çağrılır: `c1.add(c2)`. Artık tek parametre yeterlidir. Metodu çağıran nesne `this`'tir, parametre olarak gelen ise öteki sayıdır. Sonuç yine yeni bir nesne olarak döndürüldüğü için dönüş tipi `Complex` olmak zorundadır.

Hoca aynı metodun `c2.add(c1)` biçiminde de çağrılabileceğini gösterdi. Bu durumda `c4` ile `c5`'in içerikleri tamamen aynıdır, ama farklı adreslerde oturan farklı nesnelerdir. Yukarıdaki çıktıda bu görülür: `c4 == c5` `false`, `c4.equals(c5)` `true` verir.

2.5 Üçüncü çözüm: nesnenin kendisini değiştirmek

Hoca başka nasıl bir tasarım yapılabileceğini sordu. Bir öğrenci, sonucun doğrudan çağrılan nesne üzerinde tutulabileceğini söyledi. Bu durumda yeni nesne üretilmez:

```
public Complex add(Complex c1) {
    this.real += c1.real;
    this.imaginary += c1.imaginary;
    return this;
}
```

Bu kez orijinal nesnenin içeriği değişir. Hoca bunu aynı sırayla gösterdi ve bir öğrencinin önerisiyle önce değerleri yazdırıp işlemleri sonra yaptı, çünkü değişim böyle daha net görülüyordu.

1. Başlangıç: $c_1 = 1 + 2i$, $c_2 = 2 + 2i$.
2. `c3 = Complex.add(c1, c2)` yeni nesne üretir: $c_3 = 3 + 4i$. `c1` ve `c2` değişmez.
3. `c4 = c1.add(c2)`, `c1`'i değiştirir: $1+2 = 3$, $2+2 = 4$. Artık $c_1 = 3+4i$ ve `c4` aynı nesneyi gösterir.
4. `c5 = c2.add(c1)`, `c2`'yi `c1`'in güncel haliyle değiştirir: $2+3 = 5$, $2+4 = 6$. Artık $c_2 = 5+6i$ ve `c5` aynı nesneyi gösterir.

Bu metotla aynı sırayla çalıştırılan kodda `c1`'den `c5`'e kadar sırasıyla `3 + 4i`, `5 + 6i`, `3 + 4i`, `3 + 4i`, `5 + 6i` yazdı; `c4 == c1` ve `c5 == c2` de `true` çıktı (javac ile derlenip çalıştırıldı, toplamlar python ile hesaplandı).

Hoca bir yol daha saydı: `public Complex add(Complex c1, Complex c2)` metodu statik olmadan da yazılabilir. O zaman metot bir nesne üzerinden çağrılır ama `this` hiç kullanılmaz, yalnız iki parametre kullanılır. Yani o nesne boş yere üretilmiş olur.

> **Ders sonrası not:** Bu statik olmayan iki parametreli sürüm, statik `add(Complex, Complex)` ile aynı sınıfta durmaz, çünkü imzaları aynıdır: `method add(Complex,Complex) is already defined in class Complex` (javac ile denendi). Hoca bunu ayrı bir tasarım olarak anlattı, ikisini birlikte yazmak gerekmez.

Bütün bu çözümlerde ortak olan şudur: Java'da hiçbir zaman `+` operatörünü kullanamıyoruz.

3. C#'ta Operatör Tanımlama

Hoca aynı sınıfı C#'ta yazdı. Java ile benzediği için kod büyük oranda kopyalandı, adı `MyComplex` oldu.

Operatör tanımlarken kurallar şunlardır:

- Metot `public` olur ve `static` olmak zorundadır.
- `operator` anahtar kelimesi yazılır, ardından tanımlanacak operatör gelir.
- Bir şey döndürmek zorundadır. Dönüş tipi kurguya bağlıdır. Burada iki karmaşık sayının toplamı karmaşık sayı olduğu için `MyComplex` döndürülür, istense `int` de döndürülebilirdi.

Metodun içeriği, Java'da yazdığımız `add` ile hemen hemen aynıdır. Fark, artık `c1 + c2` yazabilmemizdir. Hocanın sözüyle, dil böyle bir imkan tanıyorsa bu gayet güzel bir özelliktir, desteklemiyorsa yapacak bir şey yoktur.

```
using System;

class MyComplex {
    public int real, imaginary;

    public MyComplex(int real = 0, int imaginary = 0) {
        this.real = real;
        this.imaginary = imaginary;
    }

    public static MyComplex operator +(MyComplex c1, MyComplex c2) {
        MyComplex c = new MyComplex();
        c.real = c1.real + c2.real;
        c.imaginary = c1.imaginary + c2.imaginary;
        return c;
    }

    public static bool operator ==(MyComplex c1, MyComplex c2) {
        return c1.real == c2.real && c1.imaginary == c2.imaginary;
    }

    public static bool operator !=(MyComplex c1, MyComplex c2) {
        return !(c1 == c2);
    }

    public override string ToString() {
        return real + " + " + imaginary + "i";
    }
}

class Program {
    static void Main() {
```

```
        MyComplex c1 = new MyComplex(1, 2);
        MyComplex c2 = new MyComplex();
        MyComplex c3 = c1 + c2;
        Console.WriteLine(c1);
        Console.WriteLine(c2);
        Console.WriteLine(c3);
        Console.WriteLine(c1 == c1);
        Console.WriteLine(c1 == c2);
        Console.WriteLine(c1 != c1);
    }
}
```

Çıktı (dotnet ile derlenip çalıştırıldı):

```
1 + 2i
0 + 0i
1 + 2i
True
False
False
```

Sayısal olarak: $c_1 = 1 + 2i$ ve varsayılan parametrelerle $c_2 = 0 + 0i$ olduğundan $c_3 = (1 + 0) + (2 + 0)i = 1 + 2i$ olur.

Yazarken iki küçük şey görüldü. Birincisi, hoca parametresiz constructor'dan diğerini `this(0, 0)` ile çağıramadı, sözdizimini o an hatırlayamadığı için varsayılan parametre (default parameter) kullandı. İkincisi, `ToString` yazılırken Java'daki gibi bir annotation değil, `override` anahtar kelimesi gerekiyor. Bu kelime yazılmadan çalıştırılınca ekranda varsayılan `ToString`'in çıktısı göründü.

> **Ders sonrası not:** Hoca C#'ta constructor çağırma sözdizimini derste hatırlayamadı; doğrusu `public MyComplex() : this(0, 0) { }` biçimidir. Aynı yapı denenen küçük bir sınıfta derlenip çalıştı (dotnet ile denendi).

3.1 == ve != birlikte tanımlanmalıdır

Hoca ardından eşitlik operatörünü aşırı yükledi. Burada iki noktaya dikkat etmek gerekir. Birincisi, == bir `MyComplex` değil, `bool` döndürür. Eşitliğin ne demek olduğu tamamen bize aittir, burada gerçel ve sanal kısımların ikisinin de eşit olması istendi.

İkincisi, bu tanım tek başına hata verir. C#, bir operatörü tanımladıysak onun karşılığını da yazmamızı ister. Yani == tanımlandıysa != de tanımlanmak zorundadır. Hoca !='yi =='yi çağırıp sonucun tersini alarak yazdı. Toplamada böyle bir zorunluluk yoktu.

Derleyici mesajı: `The operator 'M.operator ==(M, M)' requires a matching operator '!=' to also be defined` (dotnet ile denendi).

Hocanın denemesi şöyleydi: `c1 == c1` `true`, `c1 == c2` ve `c1 != c1` `false` verdi. Çıktıda da bu görülüyor.

Soru: Operatörleri override ederken neden ek bir şey belirtmemiz gerekmiyor? **Cevap:** Burada override etmiyoruz, overload ediyoruz. Override, kalıtımın olduğu yerde olur: soyut bir sınıftan kalıtım alırken, bir interface'i implement ederken ya da normal bir sınıfın metodunu geçersiz kılarken.

> **Hocanın vurgusu:** Overload ile override'ı karıştırmayın. Operatör aşırı yükleme bir overload'dur. Override kalıtımın olduğu yerde olur.

4. C++, Python ve Operatör Aşırı Yüklemenin Sınırları

Hoca ekranda başka dillerin belgelerini açarak farkları gösterdi.

C++: Kutu (`Box`) örneğinde operatör bir `const` referans alır ve yine `Box` döndürür:

```
Box operator+(const Box& b);
```

Statik olması gerekmez. Metot belli bir nesne üzerinden çalıştığı için `this` ile tanımlanabilir. C#'tan farkı budur: C#'ta statik olmak zorundadır. Ayrıca C++'ta referans mekanizması olduğu için işlem referans üzerinden yapılır.

Python: `add` adlı bir metot yazılır, ama adının başında ve sonunda iki alt çizgi vardır: `__add__`. Python'daki bu gösterim operatörü aşırı yüklemek demektir. `p1 + p2` yazıldığında arka planda otomatik olarak bu metot çağrılır. Aynısı karşılaştırma operatörleri için de vardır: `__lt__`, `__le__`, `__gt__`, `__ge__`.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __lt__(self, other):
        return (self.x, self.y) < (other.x, other.y)
```

Ekrandaki `Point` örneğinin alanları ve `__lt__` kuralı altyazıda okunmadığı için yukarıdaki gövdeler yer tutucudur, yalnız mekanizmayı gösterir; python ile çalıştırıldı.

> **Ders sonrası not:** Hoca `__lt__` için "lower than" dedi; doğrusu büyük olasılıkla "less than"dır. Hoca da sonraki metotları "less than or equal, greater than" diye okudu.

Operatör aşırı yükleme dilden dile değişir. Her operatör aşırı yüklenemez, dilin listesine bakmak gerekir.

Soru: Bir operatörü aşırı yüklemek değil de yeni bir operatör tanımlamak mümkün mü? Örneğin `K` harfini operatör olarak tanımlayabilir miyiz? **Cevap:** Hayır. Operatör tanımlamak dilin temellerine ait bir şeydir. Burada yalnızca var olan operatörlerin aşırı yüklenmesi anlatılıyor, çünkü aşırı yükleme mekanizması nesneye yönelik programlamanın bir özelliğidir.

5. Çoklu Kalıtım

Nesneye yönelik programlamaya ait olup Java'da bulunmayan bir diğer kavram çoklu kalıtımdır. Python'da bir sınıf iki üst sınıftan birden kalıtım alabilir:

```
class Base1:
    pass

class Base2:
    pass

class MultiDerived(Base1, Base2):
    pass
```

Örnekteki `pass`, gövdeyi boş bırakmak içindir, iskelet kurmaya yarar. Kod python ile çalıştırıldı.

Java'da ise iki kavramı ayırmak gerekir:

- **Çok seviyeli kalıtım (multilevel):** `Base`'den `Derived1`, `Derived1`'den `Derived2` türer. Bu Java'da mümkündür.
- **Çoklu kalıtım (multiple):** Bir sınıfın iki üst sınıfı vardır. Bu Java'da mümkün değildir. Birden fazla üst tip almanın Java'daki yolu, dersin ikinci kısmında görülecek olan interface'leri `implements` etmektir.

`public class C extends B1, B2` satırı Java'da `'{' expected` hatası verir (javac ile denendi).

Hoca, C#'ta ve bildiği kadarıyla C++'ta da çoklu kalıtım olmadığını söyledi, sonra ekranda aratıp C++'ta var olduğunu gördü ve düzeltti: `Rectangle` sınıfı iki üst sınıftan birden kalıtım alıyordu. İki üst sınıftan türeyen bir C++ sınıfı `g++` ile de derlendi.

Hoca bu noktada nesneye yönelik programlamada göreceğimiz kavramların yaklaşık %90'ının bittiğini söyledi. Geriye interface kavramı ve generic'ler kaldı.

6. Nesnelerde == ve equals

Aradan dönünce hoca, Java kodlarında öğrencilerin çok yaptığı bir hatayı hatırlattı: İki nesne için `c1 == c2` yazmak.

Soru: `c1 == c2` ne sonuç verir, `true` mu, `false` mu? **Cevap:** Bir öğrenci her zaman `false` döneceğini, çünkü nesnelerin tutulduğu yerlerin karşılaştırıldığını ve bu yerlerin hiçbir zaman aynı olamayacağını söyledi. Hoca referansların karşılaştırıldığı kısmını doğruladı, ama "hiçbir zaman" kısmını örnekle düzeltti: `c1` ile `c2` farklı nesneleri gösteriyorsa sonuç `false`'tur. Ama `c2 = c1;` yazarsak ikisi de aynı nesneyi gösterir ve sonuç `true` olur.

Bu atamadan sonra `c2`'nin eski gösterdiği nesneye kimse bakmaz. Hocanın ifadesiyle orası bir müddet sonra kaybolur, ölür gider.

> **Hocanın vurgusu:** Nesnelerde == referansları karşılaştırır. Bu, çok yapılan bir hatadır. İçerik karşılaştırmak için fonksiyon yazılır.

İçeriği karşılaştırmak için `equals` metodu kullanılır. O da `toString` gibi `Object`'ten gelir ve override edilir. Hoca NetBeans'te "Insert Code" menüsünden `equals` ürettirdi ve kodu satır satır okudu. Kod, 2.3'teki sınıfın içinde yer alır. Akış şöyledir:

1. `this == obj` ise, yani gelen referans bu nesnenin kendisiyse, doğrudan `true` döner.
2. `obj null` ise `false` döner.
3. `getClass()` ile iki nesnenin sınıf bilgisi karşılaştırılır. Farklıysa `false` döner.
4. Parametre `Object` türünde geldiği için `Complex`'e cast edilir.
5. Üyeler tek tek karşılaştırılır. `real` veya `imaginary` farklıysa `false` döner.
6. Hiçbirine takılmadıysa `true` döner.

Hoca, parametre `Object` olduğu için her türden nesnenin gönderilebildiğini, istenirse buraya `Complex` yazılarak sınırlanabileceğini söyledi.

> **Ders sonrası not:** Parametre tipi `Complex` yapılırsa metot artık `Object.equals`'ı override etmez, overload eder. Üstünde `@Override` varsa `method does not override or implement a method from a supertype` hatası alınır (javac ile denendi). Bu yüzden override için parametrenin `Object` kalması gerekir.
>
> **Ek bilgi (derste anlatılmadı):** 2.3'teki sınıf `-Xlint:all` ile derlenince javac, `equals` override edilip `hashCode` edilmediği için `[overrides]` uyarısı verdi (javac ile denendi). Bu bir hata değildir, kod derlenir ve çalışır. C# örneğinde de == tanımlanıp `Equals` ve `GetHashCode` override edilmediği için `CS0660` ve `CS0661` uyarıları görüldü (dotnet ile denendi).

7. Interface Kelimesinin Gündelik Anlamı

Hoca konuya nesneye yönelik programlamadan ve Java'dan bağımsız bir soruyla girdi: Interface, yani arayüz denince aklınıza ne geliyor?

Soru: Interface kavramı hakkında ne biliyorsunuz? **Cevap:** Bir öğrenci önce Swift'teki protocol kavramını anlattı: Bir sınıf ya da struct için iskelet gibidir, değişken ve fonksiyonları tanımlar. Bu protocol'e uyan yapı, içinde tanımlananların hepsine sahip olmak zorundadır. Hoca bunun nesneye yönelik programlamadaki kavramın iyi bir açıklaması olduğunu, ama kendisinin kelimenin genel anlamını sorduğunu söyledi.

Başka bir öğrenci şöyle cevap verdi: Arka planda hesaplamalar ve işlemler olur, arayüz bunlara daha kolay ve etkin ulaşmayı sağlar. Hoca bunu örneklerle açtı. Bir monitör ya da bir yazılım düşünelim. Kullanıcı olarak arkasındaki bütün koda veya donanıma hakim olmak zorunda değiliz. Monitörden görüntü almak için tuşları kullanırız. Elektron tüpünün ya da LCD'lerin davranışıyla ilgili bir ayar yapmayız.

Benzer bir örnek de grafik kullanıcı arayüzüdür (GUI).

Soru: Grafik arayüzün amacı nedir? **Cevap:** Kullanıcıya daha elverişli bir hizmet sağlamak. Kullanıcının programın yaptığı her şeyi görmesine gerek yoktur, yalnız ihtiyaç duyduğu şeyleri verimli biçimde yapması yeterlidir. Hoca buna şunu ekledi: Her şeyi göremez de zaten.

Hocanın örneği bir banka sistemiydi. Birinci sınıfta toplama, çıkarma, çarpma ve bölme yapan menüler yazarız. Banka sisteminde aynı yöntemle her fonksiyon için bir seçenek koysak 500 maddelik bir menü çıkardı. Onun yerine arayüzde menüler ve alt menüler üzerinden bütün fonksiyonları kullanabiliyoruz.

8. Java'da Interface: Tanım, UML ve implements

Nesneye yönelik programlamada interface, üyesi olmayan bir soyut sınıf gibi düşünülebilir. Sınıfa benzer, ama içine normal üye alan eklenemez. İstenirse yalnızca `public final static` alanlar eklenebilir. Hocanın tanımıyla interface, metotlardan oluşan isimli bir koleksiyondur (a named collection of methods). İçine metotların gövdesi değil, imzaları (prototipleri) yazılır ve her imzanın sonuna noktalı virgül konur.

Java'nın yeni sürümlerinde `default` metotlar da yazılabilir, yani bir metodun varsayılan bir gerçeklemesi interface'in içinde verilebilir. Hoca slaytlarda bunun anlatılmadığını söyledi.

> **Ek bilgi (derste anlatılmadı):** `default` metodun gövdesi vardır ve sınıfın onu yazması zorunlu değildir. `public interface I { int a = 5; default int f() { return a; } }` derlendi (javac ile denendi). Aynı denemede `a` alanının `public static final` olduğu görüldü, yani bu değiştiriciler yazılmasa da alan sabit sayılır.

UML gösterimi: Soyut sınıflarda ad italik yazılır ya da `abstract` notasyonu eklenir. Interface'te ise `<<interface>>` notasyonu eklenir. Bölmelerde önce varsa `public final static` alanlar, sonra metot imzaları yer alır. Okun biçimi de ayırt edicidir:

- **Kalıtım:** Düz çizgili ok.
- **Interface gerçekleştirme:** Kesikli çizgili ok.

Bir sınıf interface'i `implements` anahtar kelimesiyle alır. Soyut sınıflarda `extends` ile yalnızca bir üst sınıf alınabiliyordu, `implements` ile birden fazla interface yazılabilir. Hoca en önemli farkları Türkçe olarak tekrarladı: Yalnızca metotların tanımları yazılır ve `implements` edilen interface'lerdeki bütün metotların, `default` olanlar hariç, gerçeklemesi yazılmak zorundadır.

```
class B { }
interface I1 { }
interface I2 { }

public class C extends B implements I1, I2 { }
```

Bu satırlar derlenir (javac ile denendi). `extends` ile bir sınıf, `implements` ile birden fazla interface alınır.

> **Ders sonrası not:** Hoca interface metotları için "bunlar soyut metot değil" dedi; doğrusu gövdesiz interface metotları soyuttur, yalnız `abstract` kelimesini yazmak gerekmez. `IDatabase`'in üç metodunun da değiştiricisi çalışma zamanında `public abstract` çıktı (javac ile derlenip denendi). Ayrıca "bütün metotları yazmak" soyut sınıftan farklı bir kural değildir: Somut bir sınıf soyut sınıftan kalıtım alırken de soyut metotları yazmak zorundadır, yazmazsa aynı hata alınır. Tersine `abstract` olarak işaretlenmiş bir sınıf, interface'i gövde yazmadan `implements` edebilir (ikisi de javac ile denendi).

Hoca bu zorunluluğun anlamını şöyle anlattı: Bu bize standartlara uymayı sağlar. Özellikle generic'lerde göreceğimiz sınıf hiyerarşilerinde, kendi tasarlamadığımız bir sınıf üstte bir yerden kalıtım alıyor, bir yerden interface alıyor, ağaç gibi çok seviyeli bir yapı oluşuyor. Kendi sınıfımızı o yapının altına yerleştirirsek üstten gelen zorunlu metotları yazmamız gerekir. Hoca bunun Big Data tarafında Hadoop kütüphanesinde ve thread'lerde çok kullanılacağını söyledi. Yazılmazsa kod derlenmez.

> **Hocanın vurgusu:** Interface'i implement eden sınıf, içindeki bütün metotları yazmak zorundadır. Bu zorunluluk bir standart sağlar. Yazmazsanız derleme zamanı hatası alırsınız.

9. Interface Neden Kullanılır, Kuralları Nelerdir?

Hoca interface kullanmanın üç amacını saydı.

- **Sorumlulukları gruplamak:** Bir yazıcı nesnesinin pek çok sorumluluğu vardır. Bunlar `IPrinter` gibi bir interface'te toplanabilir, örneğin `print` ve `updateDriver`. Sorumluluklar, yani metotlar, böylece gruplanmış olur.
- **Nesnelere birden fazla görünüm vermek:** Hocanın "çok önemli" dediği amaçtır. Bir veritabanı sınıfı düşünelim. Şu an Oracle kullanıyoruz, beş yıl sonra SQLite'a geçebiliriz. Interface kullanmıyorsak bütün tanımları baştan değiştirmemiz gerekir. Interface sayesinde yalnızca yeni görünümün adını değiştiririz. Her veritabanında ekleme, güncelleme ve silme vardır. Bir kitabı eklemek için yapılacaklar bellidir, yalnız arka plandaki teknoloji değişir.
- **Kalıtım yerine hafif bir ilişki kurmak:** Kalıtım ağır (heavyweight), interface hafif (lightweight) bir ilişkidir. Kalıtım yalnızca gerçekten gerektiğinde kullanılmalıdır. Çoklu kalıtım yerine de interface kullanılır.

Kurallar şunlardır:

- Sınıf, implement ettiği interface'in bütün metotlarının gövdesini yazmalıdır.
- Normal üye alan tanımlanamaz. `int a;` derlenmez, `= expected` hatası verir (javac ile denendi). Yalnızca `public final static` alanlar olur.
- Constructor yazılamaz. Denemede `<identifier> expected` hatası alındı (javac ile denendi).
- Bir sınıf birden fazla interface implement edebilir.
- Adlandırma geleneği olarak interface adları büyük `I` ile başlar. `IDatabase` adını görünce bunun bir interface olduğunu ve veritabanı için genel bir kavram olduğunu anlarız.

10. Slayt Örneği: Araç Vergisi

Hoca interface tasarımını slayttaki bir gereksinim analiziyle gösterdi: araçların vergisini hesaplamak. Bir vergi ödeme sistemi deyince akla gelen olası her şey bir standart gibidir, yapacağımız şey sistemi bu standartlara uygun hale getirmektir. Gereksinimler şöyleydi:

- Motosiklet, araba ve otobüs ticari araç olarak kaydedilebilir.
- Yalnızca motosiklet ve araba kişisel araç olarak kaydedilebilir.
- Yalnızca ticari araçların vergisi amortize edilebilir.
- Ticari olsun olmasın, farklı araçların vergi hesabı birbirinden çok farklıdır.
- Metotlar zorunludur, ama gerçeklemeleri her araçta farklıdır. Bu, soyut metotlarda ve çok biçimlilikte gördüğümüz duruma benzer.

Yapı düz metinle şöyle gösterilebilir:

```
<<interface>> ICommercialVehicle : calculateAmortizedTax, getPlate
<<interface>> IPersonalVehicle   : calculateTax, getPlate

Motorcycle  implements ICommercialVehicle, IPersonalVehicle
Car         implements ICommercialVehicle, IPersonalVehicle
Bus         implements ICommercialVehicle
```

Bir interface'ten nesne üretilemez. Yalnızca referans alınabilir. Bu nedenle kutulardaki sınıf benzeri gösterimler nesne değildir. Sınıflar ise interface'lerin özelliklerini çoklu kalıtıma benzer bir yolla alır.

Araba sınıfı: Alanları plaka, model yılı ve motor hacmidir, bir constructor'ı vardır. `calculateTax` taban vergiyi ve motor hacmini kullanır. `calculateAmortizedTax` taban vergiyi ve içinde bulunulan yılı kullanır, aracın yaşı 10'dan küçükse bir formül, değilse başka bir satır çalışır. `getPlate` her iki interface'ten de gelir. Bunlara ek olarak model yılı ve motor hacmi için get metotları vardır. Zorunlu metotlar yazılmazsa derleme zamanı hatası alınır.

Otobüs sınıfı: Yalnızca ticari araç interface'ini implement eder. Bu yüzden vergi metotlarından yalnızca `calculateAmortizedTax`'ı içerir.

Aşağıdaki iskelet yalnız bu yapıyı gösterir. Parametresiz imzalar ve `double` tipi yer tutucudur. `getPlate`'in üst interface'e çıkarıldığı son hali verilmiştir (javac ile derlendi):

```java
interface IVehicle {
    String getPlate();
}

interface ICommercialVehicle extends IVehicle {
    double calculateAmortizedTax();
}

interface IPersonalVehicle extends IVehicle {
    double calculateTax();
}

class Car implements ICommercialVehicle, IPersonalVehicle {
    private String plate;

    public String getPlate() { return plate; }            // tek gövde iki interface'i de karş
    public double calculateTax() { return 0; }            // formül slaytta
    public double calculateAmortizedTax() { return 0; }   // formül slaytta
}

class Bus implements ICommercialVehicle {
    private String plate;

    public String getPlate() { return plate; }
    public double calculateAmortizedTax() { return 0; }   // formül slaytta
}
```

`VehicleRegistrationSystem` sınıfı: Ticari araçları ve kişisel araçları iki ayrı listede tutar. Bir aracın kişisel mi ticari mi olduğu bir kapsayıcı nesnede, yani bir listede saklanır. Metotları ticari ve kişisel araç kaydetme, arama ve kayıt silmedir. Slayt, bu metotların mantığının nasıl kurulacağını soru olarak bırakır, derste kodu yazılmadı.

Interface mi, soyut sınıf mı? Farklı araçların vergi hesabı birbirine benzeseydi ya da parametrelerle ifade edilebilseydi, interface yerine bir soyut taban sınıf daha iyi bir seçim olurdu. Ticari ve kişisel araçların hesabı benzer olsaydı da aynı şey geçerliydi. Burada hesaplar farklı olduğu için interface seçildi. Slayt bu durumları alıştırma olarak bırakıyor.

Interface'ler arasında kalıtım: `getPlate` iki interface'te de tekrar ediyor. Bir interface başka bir interface'ten kalıtım alabildiği için bu metot bir üst interface'e çıkarılır. Böylece aynı tanımı her yerde yazmaya gerek kalmaz.

11. Kod Örneği: IDatabase

Hoca interface'in asıl kazancını kısa bir örnekle gösterdi. Yeni bir pakette `IDatabase` interface'i oluşturuldu. Veritabanlarında ekleme, silme, güncelleme ve kayıt arama işlemleri vardır. Kurgu şöyleydi: `search` kitap adı alıp `Book` döndürür, `delete` verilen addaki bütün kitapları siler ve kaç kitap silindiğini döndürür, `insert` bir kitap alır.

Büyük bir şirket olduğumuzu ve şu anda Oracle üzerinde çalıştığımızı düşünelim. `OracleDB implements IDatabase` dediğimiz anda bu standartlara uyduğumuzu garanti etmiş oluruz. Bunu yazar yazmaz IDE "OracleDB is not abstract" hatası verir ve metotları implement etmemizi ister. Tıklanınca metotlar otomatik eklenir. javac'ta da mesaj aynıdır: `OracleDB is not abstract and does not override abstract method delete(String) in IDatabase` (javac ile denendi).

İkinci sınıf `MsSqlDatabase` oluşturulurken NetBeans sihirbazında interface seçilebiliyor, sınıf iskeleti ona göre üretiliyor. Her iki sınıfta da gerçek veritabanı kodu yerine ekrana yazan satırlar kondu.

> **Ders sonrası not:** Hoca IDE'nin eklediği metotlar için "exception oluşmasın diye bir şeyler ekliyor" dedi; NetBeans'in eklediği satır büyük olasılıkla tersine bir `UnsupportedOperationException` fırlatan yer tutucudur. Bu yüzden gerçek gövde yazılınca o satır silinmelidir.

```java
public class Book {
    public String name;

    @Override
    public String toString() {
        return "Book{" + name + "}";
    }
}
public interface IDatabase {
    public Book search(String name);
    public int delete(String name);
    public void insert(Book b);
}
public class OracleDB implements IDatabase {
    @Override
    public Book search(String name) {
        System.out.println("Oracle searching");
        return null;
    }

    @Override
    public int delete(String name) {
        System.out.println("Oracle deleting");
        return 0;
    }

    @Override
    public void insert(Book b) {
        System.out.println("Oracle insert");
    }
}
public class MsSqlDatabase implements IDatabase {
    @Override
    public Book search(String name) {
        System.out.println("MS SQL searching");
        return null;
    }

    @Override
    public int delete(String name) {
        System.out.println("MS SQL deleting");
        return 0;
    }

    @Override
    public void insert(Book b) {
        System.out.println("MS SQL insert");
    }
}
public class Test {
    public static void main(String[] args) {
        IDatabase r = new OracleDB();
        r.delete("Kitap");

        r = new MsSqlDatabase();
        r.delete("Kitap");
    }
}
```

Çıktı (javac ile derlenip çalıştırıldı):

```
Oracle deleting
MS SQL deleting
```

`IDatabase r = new OracleDB();` satırında `r` yalnızca bir referanstır, nesne `OracleDB`'den üretilir. `new IDatabase()` yazılamaz: `IDatabase is abstract; cannot be instantiated` (javac ile denendi).

Derste bu satır IDE'de bir süre hata gösterdi. Hoca Java'nın yeni sürümünde bir şey değişip değişmediğini kontrol etti, geçen yılın örneğine baktı ve "build" dedikten sonra hata kayboldu. Kodda hata yoktu, sorun IDE'nin geç derlemesinden kaynaklanıyordu. Aynı kod javac ile sorunsuz derlendi.

Hoca kazancı şöyle anlattı: Bin satırlık kod yazdınız, veritabanına bağlanma ve sorgulama işlemleri uzun işlerdir. Sonra "Oracle pahalı, başka bir şeye geçelim" dediniz. Yapacağınız şey yalnızca nesnenin üretildiği satırı değiştirmektir. Oracle da olsa SQL Server da olsa silinecek kitap aynıdır. Bir kitap sitesine girdiğinizde de "MS SQL'de mi arıyorsun?" diye sorulmaz. Interface, taşınabilirliği ve değişikliği çok daha rahat hale getirir, çoklu bir görünüm sağlar.

> **Hocanın vurgusu:** Asıl kazanç taşınabilirliktir. Arka plandaki teknolojiyi değiştirmek için yalnızca nesnenin üretildiği satır değişir.

Hoca ileride thread'lerde `Runnable` interface'ini göreceğimizi de söyledi. Onun `run` adlı bir soyut metodu vardır. Thread çalıştırmak için bu interface kullanılıyorsa `run` override edilmek zorundadır. `class R implements Runnable { }` satırı derlenmedi: `R is not abstract and does not override abstract method run() in Runnable` (javac ile denendi).

12. Enum'lar

12.1 Sınırlı değer kümesi

Bazı değişkenler yalnızca sınırlı bir değer kümesinden değer almalıdır. Giysi bedeni S, M, L ve X olabilir. Pizza boyu küçük, orta, büyük ve çok büyük olabilir. Bunlar kategorik değişkenlerdir.

Bu değerleri tamsayılarla kodlayabiliriz, ama değişkene 5, 6 ya da başka bir tamsayı da yazılabilir. Etiketler doğası gereği sınırlıyken onları sınırsız bir şeyle kodlamış oluruz. Bu hataya yatkın bir kurulumdur, 0 ya da başka geçersiz değerler programda soruna yol açabilir.

12.2 Java'da enum

```java
public enum Size {
    SMALL, MEDIUM, LARGE, EXTRA_LARGE
}
```

Burada yalnızca etiketler vardır. Kullanımı şöyledir:

```
Size s = Size.MEDIUM;
```

`Size` değişkenin tipi, `s` bu tipten bir referans, `Size.MEDIUM` ise değeridir. Aslında burada bir sınıf tanımlamış olduk. Adı `Size`'dır ve bu sınıftan yalnızca dört statik örnek üretilebilir. `Size s = MEDIUM;` ya da `Size s = M;` yazılamaz, önek zorunludur.

Özetle:

- Enum bir sınıf tanımıdır.
- Enum'un her üyesi o sınıfın bir örneğidir.
- Tanımlı olanların dışında başka üye olamaz. `new Size()` denemesi `enum classes may not be instantiated` hatası verdi (javac ile denendi).
- Normal sınıflar gibi alanları, metotları ve constructor'ları olabilir.

> **Ders sonrası not:** Slaytta "C'nin aksine karşılık gelen tamsayı yok" deniyor. Hoca da C'de 0, 1, 2 diye otomatik değer verildiğini, burada böyle olmadığını anlattı. Enum sabiti bir tamsayı değildir, `Size s = 2;` gibi bir atama yapılamaz. Ama her sabitin sırası vardır: `Tariff.NET4.ordinal()` 1 döndürdü (javac ile derlenip çalıştırıldı).

12.3 Alanları olan enum: Tariff

Hoca slaytta tarife örneğini gösterdi. Burada yine sabitler vardır, ama her biri constructor'a değer gönderir. Sabitler `SMALL`, `MEDIUM` gibi düşünülür, ayrıca kota, hız ve ücret alanlarına değer atarlar. Bu yüzden oldukça kullanışlıdır: Hangi tarife seçilirse alanlar da ona göre belirlenir, ayrı ayrı etiket vermek gerekmez.

```
public enum Tariff {
    NETFREE(0, 4, 60), NET4(4, 8, 30), NET6(6, 8, 40);   // sabitler ilk satırda olmak zorunda

    private int quota, speed, fee;

    private Tariff(int quota, int speed, int fee) {
        this.quota = quota;
        this.speed = speed;
        this.fee = fee;
    }

    public int getQuota() { return quota; }
    public int getSpeed() { return speed; }
    public int getFee()   { return fee; }
    public void setFee(int fee) { this.fee = fee; }

    @Override
    public String toString() {
        return name() + "[quota=" + quota + ", speed=" + speed + ", fee=" + fee + "]";
    }
}
```

Hoca iki kuralın altını çizdi:

- Sabitler ilk satırda olmak zorundadır. Önce alan yazınca `enum constant expected here` hatası alındı (javac ile denendi).
- Constructor `private` olmak zorundadır.

  Ders sonrası not: Hoca constructor'ın `private` tanımlanması gerektiğini söyledi. Tamamlayıcı ayrıntı şudur: Erişim belirteci hiç yazılmazsa kod derlenir, `public` ya da `protected` yazılırsa `modifier public not allowed here` benzeri hata alınır (javac ile denendi).

12.4 Person ve Subscription

Slaytta `Person` ile `Subscription` arasında birebir bir ilişki var.

Soru: Kişi ile üyelik arasındaki ilişkinin adı nedir? Kardinalite değil, ilişkinin adı soruluyor. **Cevap:** Association. Okun biçiminden anlaşılır: Dependency olsaydı kesikli çizgi olurdu. Burada parça bütün ilişkisinden de söz edemeyiz, çünkü durum birebirdir.

Her üyeliğin kendi tarifesi vardır. `Person` bir kişi adı ve üyelik bilgisi tutar. Üyelik için yeni bir `Subscription` oluşturulur ve ona tarife gönderilir, referansı göndermek yeterlidir. `Subscription` tarifeyi ve bir tarih tutar: Nesne üretildiğinde hangi tarifeye eşit olduğu atanır, o anın tarihi de alınır. `toString` çağrıları sırayla iç içe gider.

Aşağıdaki kod anlatımdan kurulmuştur. Kişi adları yer tutucudur. Derste gösterilen set çağrısının hangi alanı değiştirdiği net olmadığından burada `setFee` yazılmıştır.

```
import java.util.Date;

public class Subscription {
    private Tariff tariff;
    private Date date;

    public Subscription(Tariff tariff) {
        this.tariff = tariff;
```

```
        this.date = new Date();   // oluşturulduğu anın tarihi
    }

    public Tariff getTariff() { return tariff; }

    @Override
    public String toString() {
        return tariff.toString();
    }
}

public class Person {
    private String name;
    private Subscription subscription;

    public Person(String name) {
        this.name = name;
    }

    public void subscribe(Tariff tariff) {
        subscription = new Subscription(tariff);
    }

    @Override
    public String toString() {
        return name + ": " + subscription;
    }
}

public class TestEnum {
    public static void main(String[] args) {
        Size s = Size.MEDIUM;
        System.out.println(s);

        Tariff t = Tariff.NET4;
        Person p1 = new Person("Kisi A");
        p1.subscribe(t);
        Person p2 = new Person("Kisi B");
        p2.subscribe(Tariff.NETFREE);
        System.out.println(p1);
        System.out.println(p2);

        t.setFee(35);
        System.out.println(p1);
        System.out.println(Tariff.NET4.getFee());
        System.out.println(Tariff.NET4.ordinal());
    }
}
```

Çıktı (javac ile derlenip çalıştırıldı):

```
MEDIUM
Kisi A: NET4[quota=4, speed=8, fee=30]
Kisi B: NETFREE[quota=0, speed=4, fee=60]
Kisi A: NET4[quota=4, speed=8, fee=35]
35
1
```

Hoca enum'ları basit ama bazı durumlarda işe yarayan bir yapı olarak özetledi.

> **Ek bilgi (derste anlatılmadı):** Hoca enum'daki değerlerin sonradan set edilebildiğini gösterdi. Burada dikkat edilecek nokta, sabitin programda tek bir örneği olmasıdır: Yukarıdaki kodda `t.setFee(35)` satırından sonra `Tariff.NET4.getFee()` de

35 döndürdü (javac ile derlenip çalıştırıldı). Yani değişiklik o tarifeyi kullanan bütün üyelikleri etkiler.

13. Generic Sınıflar ve Generic Metotlar

13.1 Neden generic?

Generic sınıf, generic programlamadan gelir ve bir kodu farklı veri türleri için tekrar tekrar kullanmayı sağlar. Hoca ihtiyacı overloading üzerinden anlattı. Overloading bir metodu aynı adla farklı imzalarla yazmayı sağlıyordu, ama aynı şeyi 500 kez overload etmek gereksiz bir yük oluşturur. Örneğin ekrana bir dizinin elemanlarını yazdıracağız. Dizi `int` dizisi olabilir, karakter dizisi olabilir. Her biri için ayrı fonksiyon yazmak yerine generic bir fonksiyon ya da generic bir sınıf yazarak işi tek bir tanımla çözebiliriz. Slayt bunun C++'taki template'lere benzediğini söylüyor.

Aynı durum veri yapılarında da vardır. Geçen dönem C'de kuyruk (queue) ve yığın (stack) gördük. Kuyruğu `int` için, `char` için ya da tasarladığımız bir `struct` için kullanabiliriz. Bunları ayrı ayrı yazmadan genel hale getirmenin yolu generic programlamadır.

Slayt, bu bölümün amacının kendi generic sınıfını yazmayı ya da veri yapılarını öğretmek olmadığını söylüyor. Hoca birinci kısmına katılmadığını, generic sınıfın nasıl yazıldığını göstereceğini söyledi. İkinci kısmına katıldı, çünkü veri yapıları bu dönem ayrı bir derste görülüyor.

Generic, sınıf seviyesinde de metot seviyesinde de olabilir. Bir `ArrayList` tip belirtilmeden oluşturulursa içine her türden veri eklenebilir. Yalnızca `Integer` eklemek istiyorsak tipi sınıf adından sonra açılı parantez içinde belirtiriz.

13.2 Generic sınıf yazmak

Tanımı normal bir sınıf gibidir. Fark, sınıf adından sonra açılı parantez içinde bir tip parametresi, örneğin `T`, yazılmasıdır. Sınıfın içinde bu tipteki bir nesne geçen her yerde `T` kullanılır: alanın tipi, metodun dönüş tipi, parametre tipi.

> **Ders sonrası not:** Hoca açılı parantez için "greater than or equal than symbol" dedi; doğrusu < ve > işaretleridir, yani <T>. Hoca T'yi C'deki `define` anahtar kelimesine de benzetti. Benzerlik yalnız T'nin kod boyunca bir tipin yerine geçmesidir. Doğrusu `T` derleyicinin denetlediği bir tip parametresidir: `new GenericClass<Integer>("aa")` satırı `incompatible types: String cannot be converted to Integer` hatası verdi (javac ile denendi).

```
public class GenericClass<T> {
    private T data;

    public GenericClass(T data) {
        this.data = data;
    }

    public T write() {
        System.out.println("Ekrana yaz " + data);
        return data;
    }
}
```

Hoca test sınıfında `GenericClass<int>` yazınca "unexpected type" hatası aldı ve bir süre nedenini aradı. Sonunda çözümü buldu: İlkel tipler tip parametresi olamaz, sarmalayıcı (wrapper) sınıflar kullanılır. `GenericClass<Integer>` yazılınca kod çalıştı. Mesaj javac'ta da aynıdır:

```
error: unexpected type
  required: reference
  found:    int
```

Aynı sınıf `Integer`, `String` ve `Book` ile kullanıldı. Hocanın gösterdiği şey, `write` metodunun dönüş tipinin IDE'de de seçilen tipe göre değişmesiydi: `Integer` için `Integer`, `String` için `String`, `Book` için `Book` döndürür. Üstelik `write` fonksiyonunu overload etmeye hiç gerek kalmadı.

`Book` için ekrana kitabın kendisi yazılamaz. `data` bir referanstır, yazdırılırken `Book`'un `toString` metoduna gider. Bu yüzden `Book` sınıfına bir `name` alanı ve `toString` eklendi.

```
public class TestGeneric {
    public static void main(String[] args) {
        GenericClass<Integer> g = new GenericClass<Integer>(5);
        Integer p = g.write();

        GenericClass<String> s = new GenericClass<String>("aa");
        String q = s.write();

        Book b1 = new Book();
        b1.name = "Kitap";
        GenericClass<Book> b = new GenericClass<Book>(b1);
        Book k = b.write();
    }
}
```

Çıktı (javac ile derlenip çalıştırıldı, `Book` sınıfı 11. bölümdekidir):

```
Ekrana yaz 5
Ekrana yaz aa
Ekrana yaz Book{Kitap}
```

13.3 Tip parametresi harfleri ve generic metotlar

`T` dışında yaygın harfler de vardır. Bunlar aslında birer değişken adı gibidir, ama anlamlarına göre kullanılır:

- `T`: tip (type).
- `E`: eleman (element).
- `K`: anahtar (key). Hash yapılarında görülecek.
- `V`: değer (value).
- `N`: sayı (number).

Tek bir parametre yazmak zorunda değiliz. Örneğin ikili bir yapıda bir `E` ve bir `V` alınabilir, ikisine ayrı ayrı tip verilir.

Hoca ekranda bir generic metot da gösterdi. Metot `E` türünden bir dizi alır. Aşağıda onu `Integer` dizisi ve karakter dizisi ile çağırıp ekrana basar:

```
public class GenericMethod {
    public static <E> void printArray(E[] elements) {
        for (E element : elements) {
            System.out.print(element + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Integer[] sayilar = {1, 2};
        Character[] harfler = {'a', 'b'};
        printArray(sayilar);
        printArray(harfler);
    }
}
```

Kod javac ile derlenip çalıştırıldı, dizideki değerler yer tutucudur.

Java generic'lerinde ayrıca wildcard'lar vardır, üst sınır ve sınırsız kullanımlar hangi tipin hangisinden `extends` aldığıyla ilgilidir. Hoca bunların bu dersin kapsamında olmadığını, ama bilmekte fayda olduğunu söyledi.

14. Veri Yapısı Nedir?

Veri yapısı, verileri bilgisayarın geçici belleğinde düzenleme şemasıdır. En basit örnek bir tamsayıdır: Bir tamsayıyı saklamak için `int` tipinde bir değişken kullanırız. Daha genel olarak, veri öğelerinden oluşan bir koleksiyonun bellekte nasıl düzenlendiğini anlatır.

Bu düzenleme, programın farklı işlerdeki performansını etkiler. Doğru kullanılırsa ikili ağaç, hız bakımından bağlı listeden çok daha uygundur.

Soru: İkili ağaçta eleman aramanın karmaşıklığı nedir? **Cevap:** Doğru kullanılırsa, yani ağaç dengeliyse $O(\log n)$.

> **Ders sonrası not:** "Dengeli" şartı önemlidir. Sıralı değerler sırayla eklenen bir ikili arama ağacında yükseklik eleman sayısına eşit çıktı (python ile hesaplandı). Yani böyle bir ağaç bağlı liste gibi davranır ve arama $O(n)$ olur.

Veri yapısını seçmek algoritma ve yazılım geliştirmede önemli bir adımdır. Gerektiğinde dizi yerine matris kullanmak önemlidir. Gerekmiyorsa matris kullanmak kötü programlamadır, çünkü dizi matristen daha basittir. Uygun veri yapısı algoritma ve yazılım tasarımından önce seçilmelidir.

Yaygın veri yapıları diziler, listeler, yığın, kuyruk, heap, ağaçlar ve graflardır. Bunlar veri yapıları dersinin konusudur.

15. Bağlı Liste

Bağlı liste, düğüm (node) denen ve kendine referans tutan (self-referential) sınıf nesnelerinin bir koleksiyonudur. C'de yapı elemanları arasında işaretçi kullanırız. Java'da ise bir düğümün içinde bir veri, örneğin `Book`, ve yine `Node` türünden `next` tutulur. Yıldız gibi özel bir karakter yoktur, çünkü referanslar zaten işaretçiye benzer.

```
class Node {
    Book data;
    Node next;
}
```

Bu tek bağlı listenin tipik düğümüdür. Çift bağlı listede `next` ile birlikte `prev` de tutulur.

- İlk düğüme head, son düğüme tail denir. Tail düğümü, listenin bittiğini göstermek için `null` tutar.
- Düğümlere nesne referansıyla erişilir.
- Düğümler arasında gezmek için iterator nesnesi kullanılır. İterator bir sınıftır, `while` ya da for-each döngüsüne benzer.
- İstenirse gezme kodu `while` döngüsüyle de yazılabilir, ama her veri tipi için ayrı düşünmek gerekir. Generic metotlar bu işi kolaylaştırır.
- Bütün listeyi gezmek for-each döngüsüyle daha kolaydır.

Bağlı listenin diziye göre avantajları:

- Listeyi büyütmek ya da küçültmek bir maliyet getirmez.
- Ekleme ve silme, kaydırma (shift) işlemi olmadığı için sıradan bir diziden daha hızlıdır.
- Sıralama algoritmaları, değerleri değil yalnız düğümler arasındaki bağlantıları değiştirdiği için yer değiştirme işlemlerini azaltır.

Dizinin bağlı listeye göre avantajı: Köşeli parantez ve indeks ile, yani `a[i]` biçiminde elemana doğrudan erişilir.

> **Ders sonrası not:** Slaytta "listeyi büyütmek maliyetsizdir" ve "sıralama bağlı listede daha hızlıdır" deniyor. Bunlar büyük olasılıkla koşullu ifadelerdir: Ekleme ve silmede kaydırma olmaz, ama önce konumu bulmak için liste baştan gezilir ve bu $O(n)$ sürer. Sıralamada ise veri kopyalama azalır, ama doğrudan indeks erişimi olmadığı için her algoritma aynı rahatlıkla çalışmaz.

Bağlı listenin türleri şunlardır:

- **Tek bağlı:** Her düğüm yalnız bir sonrakini gösterir.
- **Çift bağlı:** İleri ve geri gezmek için iki referans tutar.

- **Dairesel (circular):** Son düğüm listenin ilk düğümünün adresini tutar.

Hoca veri yapılarının ayrıntısını bu derste anlatmadığını, çünkü iki dersin aynı dönemde birlikte yürüdüğünü söyledi. Bu derste odak, tasarımdan çok, bunların problemlerde kullanımıdır. Yine de örneğin bağlı listeye eklemenin nasıl yapıldığını ve hangi parametrelerin gerektiğini bilmek gerekir.

> **Hocanın vurgusu:** Veri yapıları dersini almayanlar bağlı liste, çift bağlı liste ve ağaç konularına bakmalı. Bu derste önemli olan, bu yapıların tasarımı değil, Java'da kullanımıdır.

16. ArrayList

Java'da liste gerçeklemesi için `ArrayList` sınıfı kullanılır. Slayttaki özellikler "diziye dayalı, tek bağlı, thread güvenli değil" şeklindedir. Oluşturması normal nesne üretmeye benzer.

> **Ders sonrası not:** Slayttaki "single linked" ifadesi yanlıştır. `ArrayList` bir bağlı liste değildir, arkada yeniden boyutlanan bir dizi kullanır: `ArrayList`'in `RandomAccess` olduğu, `LinkedList`'in olmadığı görüldü, `LinkedList`'in düğümünde ise `next` ve `prev` alanları var (JDK 25'te çalıştırılarak denendi). Yani Java'daki `LinkedList` çift bağlıdır.

Tip belirtilmeden oluşturulursa bütün elemanlar `Object` türünden sayılır. Bu yüzden içine her şey eklenebilir: Tip belirtmeyen bir `ArrayList`'e hem bir `Integer` hem bir `String` eklenebildi, javac de `unchecked` uyarısı verdi (javac ile denendi). Generic programlama sayesinde her elemanın tipi belirtilebilir, bu da eleman tipi `Person` olan bir diziye benzer:

```
ArrayList<Person> liste = new ArrayList<Person>();

for (Person p : liste) {
    System.out.println(p);
}
```

Bu sayede `String` listesi, dosya listesi gibi ayrı sınıflara gerek kalmaz. Tek bir `ArrayList` sınıfı her sınıftan nesneyi toplar, generic programlamanın bir örneğidir. For-each dışında ileride görülecek iterator ile de gezilebilir.

Yararlı metotlar şunlardır:

- `add(e)`: Elemanı listenin sonuna ekler.
- `get(i)`: `i` konumundaki elemanı döndürür.
- `size()`: Listedeki eleman sayısını döndürür. Son elemanın konumunun bir fazlasıdır.
- `set(i, e)`: `i` konumundaki elemanı verilen elemanla değiştirir.
- `remove(i)`: `i` konumundaki elemanı siler.
- `ensureCapacity(n)`: Gerekirse kapasiteyi artırır.
- `trimToSize()`: Kapasiteyi listenin o anki boyutuna indirir, liste küçülür.

Size ile kapasite farklı şeylerdir. Kapasite, liste dolunca otomatik olarak 1,5 ile çarpılarak artar. `get` ve `remove` gibi metotlarda indeks sınır dışındaysa `IndexOutOfBoundsException` fırlatılır. Bu denetlenmeyen (unchecked) bir istisnadır, yani derleyici yakalamamızı istemez, bu yüzden çalışma sırasında programı düşürebilir.

> **Ders sonrası not:** Hoca "kapasite dışına erişilirse hata alınır" dedi; doğrusu sınır kapasite değil, size'dır. Kapasitesi 100 verilip eleman eklenmemiş bir listede `get(0)` çağrısı `Index 0 out of bounds for length 0` hatası verdi. İki elemanlı listede `get(2)` de aynı hatayı verdi, yani `i == size` da sınır dışıdır (JDK 25'te çalıştırılarak denendi).

> **Ders sonrası not:** Hoca bir yerde `ArrayList`'in varsayılan boyutu için "sıfır olabilir, bilmiyorum" dedi. `new ArrayList<>()` için size 0'dır. İç dizinin uzunluğu da başta 0 çıktı, ilk eklemede 10 oldu, sonra dolmaya bağlı olarak 15, 22, 33, 49 diye büyüdü. Bu artış yaklaşık 1,5 kattır, $10 \times 1{,}5 = 15$ ve $15 \times 1{,}5 = 22{,}5$ değeri 22'ye iner (JDK 25'te yansımayla ölçüldü).

17. Koleksiyonların Sınıf Hiyerarşisi

Hoca nesneye yönelik programlamanın "gizeminin" aslında böyle yerlerde olduğunu söyledi. Şimdiye kadar gördüğümüz interface'ler, soyut sınıflar, generic'ler ve sınıflar burada hep birliktedir. Hocanın ekranda gösterdiği yapı düz metinle şöyledir:

```
java.lang
  Iterable (interface)                 for-each ile gezmeyi sağlar
java.util
  Collection (interface, Iterable'dan kalıtım alır)
                                       add, clear, contains, isEmpty, iterator, remove, size
  List (interface, Collection'dan kalıtım alır)
                                       get, indexOf
  AbstractCollection (soyut sınıf)
    AbstractList (soyut sınıf, List'i kullanır)
      ArrayList
      Vector
      LinkedList
```

Bir `ArrayList` üzerinde kendi yazmadığımız `get` ve `indexOf` metotlarını görmemizin nedeni budur: `List` interface'inden gelirler. `List` de `Collection`'dan kalıtım aldığı için `add`, `clear`, `contains`, `isEmpty`, `iterator`, `remove` ve `size` metotları bu interface'i kullanan bütün sınıflara geçer. Bu, interface'ler arasında kalıtım olabileceğinin de bir örneğidir.

Hoca, kendi koleksiyonumuzu yazdığımızda ne olacağını anlatmak için bir örnek verdi: Yeni bir liste, örneğin `YtuList`, `AbstractList`'ten `extends` edilirse bu metotları yazmak gerekir.

> **Ders sonrası not:** Hoca `AbstractList`'ten türeyen yeni listede "bunların hepsini tanımlamak zorundayız" dedi; doğrusu `AbstractList`'te soyut kalan yalnız `get(int)` ve `size()`'dır. Yalnız bu ikisini yazan `YtuList<E>` derlendi, hiçbirini yazmayan sınıf `does not override abstract method get(int) in AbstractList` hatası verdi (javac ile denendi). Geri kalanların gövdesi soyut sınıflardan gelir.
>
> **Ders sonrası not:** Hocanın şeması kaynakta daha ayrıntılıdır. `LinkedList`'in doğrudan üst sınıfı `AbstractSequentialList`'tir, o da `AbstractList`'ten türer. Hoca da dersin sonunda bu adı andı. `AbstractCollection` ise `Iterable`'ı doğrudan değil, `Collection` üzerinden alır (JDK 25'te yansımayla denendi).
>
> **Ek bilgi (derste anlatılmadı):** JDK 25'te `List`, `Collection`'dan doğrudan değil, araya giren `SequencedCollection` üzerinden kalıtım alıyor (yansımayla denendi). Şemanın mantığı değişmez, `Collection`'ın metotları yine bütün listelere geçer.

17.1 Vector ve eskitilmiş kavramı

Hoca `ArrayList`'in "arkadaşı" `Vector`'ün bildiği kadarıyla eskitildiğini söyledi.

Soru: Deprecated ne demek? **Cevap:** Eskitilmiş, artık kullanılmayan demektir. Java'da eskitilmiş metotların adı IDE'de üstü çizili görünür.

Hoca bunu NetBeans'te `java.util.Vector` yazarak gösterdi: IDE bunun için "obsolete collection" diyordu.

> **Ders sonrası not:** Hoca `Vector` için "deprecated" dedi; doğrusu `Vector` üzerinde `@Deprecated` işareti yok, IDE'nin dediği gibi "obsolete", yani eski bir koleksiyondur. `Vector.class.isAnnotationPresent(Deprecated.class)` `false` döndürdü ve `-Xlint:all` ile derlenen bir `Vector` kullanımında uyarı çıkmadı. Aynı dosyadaki `Date.getYear()` ise `[deprecation]` uyarısı verdi (javac ile denendi).

18. Kod Örneği: Course ve Student

Hoca son örnek olarak kurs ve öğrenci uygulamasını (basit bir USIS) gösterdi. Kod Yunus Hoca'nındır. Öğrencinin numarası ve adı, kursun adı ve kodu vardır. Kurs, kendisini alan öğrencilerin listesini tutar. Dersin başındaki "neden hep dizi kullanıyoruz?" sorusunun karşılığı da budur: Artık dizi kullanmıyoruz.

Bir kurs birden fazla öğrenciden oluşabilir. Burada bir association vardır, aggregation olarak da düşünülebilir: Kurs yok edilirse öğrenciler yok olmaz.

Kurs nesnesi oluşturulduğunda listeye yeni bir `ArrayList` atanır. Kurs yalnızca bu listenin referansını tutar. `ArrayList<Student>` dinamik bir dizi gibidir ve içinde `Student` nesnelerinin referansları saklanır: Bellekte başka bir yerde üretilen nesnenin referansı listeye eklenir.

`addStudent`, kapasite doluysa ya da öğrenci zaten varsa ekleme yapmaz. Kontrol için `findStudent` kullanılır: Liste baştan sona `for` ile gezilir, her öğrencinin numarası `compareTo` ile aranan numarayla karşılaştırılır, sonuç sıfırsa öğrenci döndürülür, bulunamazsa `null` döner. `displayClassList` for-each ile bütün öğrencileri yazar.

Aşağıdaki kod, slayt kodunun anlatımdan kurulmuş halidir. Test sınıfındaki kod, numara, ad ve kapasite değerleri yer tutucudur.

```
public class Student {
    private String number;
    private String name;

    public Student(String number, String name) {
        this.number = number;
        this.name = name;
    }

    public String getNumber() { return number; }

    @Override
    public String toString() {
        return number + " " + name;
    }
}
```

```
import java.util.ArrayList;

public class Course {
    private String code;
    private String name;
    private int capacity;
    private ArrayList<Student> students;   // listenin referansı

    public Course(String code, String name, int capacity) {
        this.code = code;
        this.name = name;
        this.capacity = capacity;
        this.students = new ArrayList<Student>();
    }

    public int getNumberOfStudents() {
        return students.size();
    }

    public int getCapacity() {
        return capacity;
    }

    public boolean addStudent(Student s) {
        if (students.size() == capacity || findStudent(s.getNumber()) != null) {
            return false;   // kapasite dolu ya da öğrenci daha önce eklenmiş
        }
        students.add(s);
        return true;
    }
```

```
    public Student findStudent(String number) {
        for (int i = 0; i < students.size(); i++) {
            Student s = students.get(i);
            if (s.getNumber().compareTo(number) == 0) {
                return s;
            }
        }
        return null;
    }

    public void increaseCapacity(int newCapacity) {
        if (newCapacity < capacity) {
            return;
        }
        capacity = newCapacity;
    }

    public void displayClassList() {
        System.out.println(code + " " + name + ": " + students.size() + "/" + capacity);
        for (Student s : students) {
            System.out.println(s);
        }
    }
}

public class USIS {
    public static void main(String[] args) {
        Course c = new Course("KOD", "Ders", 2);
        System.out.println(c.addStudent(new Student("N1", "Ogrenci A")));
        System.out.println(c.addStudent(new Student("N1", "Ogrenci A")));
        System.out.println(c.addStudent(new Student("N2", "Ogrenci B")));
        System.out.println(c.addStudent(new Student("N3", "Ogrenci C")));
        c.increaseCapacity(3);
        System.out.println(c.addStudent(new Student("N3", "Ogrenci C")));
        c.displayClassList();
    }
}
```

Çıktı (javac ile derlenip çalıştırıldı):

```
true
false
true
false
true
KOD Ders: 3/3
N1 Ogrenci A
N2 Ogrenci B
N3 Ogrenci C
```

18.1 Dizili sürümle karşılaştırma

Hoca aynı örneğin daha önce dizilerle yazılan sürümüyle karşılaştırılmasını istedi. Fark en çok kapasite artırmada görülür. Geleneksel bir dizide şu adımlar gerekir:

1. Yeni, boş bir `Student` dizisi oluşturulur.
2. Eski dizi bu geçici diziye kopyalanır.
3. Geçici dizinin referansı `students` alanına kopyalanır.
4. Eski dizinin gösterdiği alan serbest kalır.
5. Kapasite değeri güncellenir.

```java
public void increaseCapacity(int newCapacity) {
    if (newCapacity < capacity) {
        return;
    }
    Student[] temp = new Student[newCapacity];   // yeni, boş dizi
    for (int i = 0; i < capacity; i++) {
        temp[i] = students[i];                   // eski elemanları kopyala
    }
    students = temp;                             // referansı yeni diziye çevir
    capacity = newCapacity;
}
```

Bu metot küçük bir sınıf içinde derlendi (javac ile denendi). Hoca `ArrayList`'te bu işi yapmadığımızı, arka planda kendisinin yaptığını söyledi.

> **Ders sonrası not:** Kurstaki `capacity` alanı `ArrayList`'in iç kapasitesi değildir, kursun kendi kontenjan sayacıdır. Yukarıdaki `increaseCapacity` yalnız bu sayacı değiştirir. Otomatik büyüyen şey `ArrayList`'in iç dizisidir, onu 16. bölümde gördük. Hoca metodu anlatırken "yeni kapasiteyi kapasiteye ekle" dedi, koddaki işlemin toplama mı atama mı olduğu altyazıdan net değil, burada atama olarak yazıldı.

18.2 LinkedList ve kaynak önerisi

Hoca `LinkedList`'te araya, başa ve sona ekleme yapılabildiğini söyledi, ama bu derste üzerinde durmadı. Belki sonra bir örnek hazırlayacağını belirtti. Önce `ArrayList` öğrenilecek.

> **Hocanın vurgusu:** `ArrayList`'in metotlarını ve sınıf hiyerarşisini bilmek zorundasınız. Geri kalanı basit şeylerdir.

Kaynak olarak javaTpoint sitesinin Java Collections bölümünü önerdi. Burada her koleksiyonun nereden kalıtım aldığı ve hangi interface'leri implement ettiği anlatılır. Örneğin `LinkedList`'in `AbstractSequentialList`'ten kalıtım aldığı, `List`, `Deque`, `Cloneable` ve `Serializable` interface'lerini implement ettiği yazar. Her metodun açıklaması ve temel örnekleri vardır. Hoca bu siteyi akademik seviyede yazılmış, güvenilir bulduğunu söyledi. Kendisi de oradan ders notu hazırlamıştır.

Dersin Toparlaması

Bu derste önce Java'da olmayan iki kavram görüldü. Java'da `+` yalnız ilkel tiplerde ve `String`'de çalıştığı için karmaşık sayı toplamak için metot yazdık. C#, C++ ve Python'da ise operatör aşırı yüklenebiliyor. Çoklu kalıtım da Java'da yok, onun yerine birden fazla interface implement ediliyor. Nesnelerde `==`'in referans karşılaştırdığını, içerik için `equals` gerektiğini de bu sırada hatırladık.

Asıl konu interface'lerdi: Metot imzalarından oluşan, nesnesi üretilemeyen, implement edene zorunluluk getiren bir standart. `IDatabase` örneği arka plandaki teknolojinin tek satırla değişebildiğini gösterdi. Enum'lar sınırlı değer kümeleri için sınıf tabanlı bir çözüm sundu. Generic'ler aynı kodu farklı tiplerle kullanmayı sağladı ve bizi `ArrayList` ile koleksiyon hiyerarşisine getirdi. Hoca sonraki derste slaytta kalınan yerden, Map yapılarından devam edileceğini, ayrıca JUnit testleriyle ilgili bazı dersler anlatacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.